

C++ Physical Computing on the Raspberry Pi 5

wiringPi Edition — a self-directed lesson plan for the Freenove Ultimate Starter Kit, plus a Sense HAT unit. Built on the maintained **WiringPi** library (the GC2 fork), which supports the Pi 5's RP1 chip.

How to use this document

This is not an answer key. Each lesson gives you a wiring target, a predict-first prompt, and a skeleton with blanks (`_____`) where you write the real logic. The boilerplate is provided so you spend your effort on the parts that teach — the loop bodies, the bit manipulation, the state tracking — not on remembering include lines. Fill the blanks, run it on hardware, then bring your code back for review.

1. Wire the circuit (Pi powered off if unsure).
2. Predict the behavior before running. Write it down.
3. Fill every `_____` yourself — no hunting for finished code.
4. Run, compare to your prediction, note surprises.
5. Review — bring the code, working or broken.

THE wiringPi-ON-PI-5 GOTCHA (read this first)

wiringPi *does* run on the Pi 5 — but **only the maintained fork**. Two dead ends will waste your afternoon:

- **Gordon Henderson's original wiringPi 2.5** was deprecated years ago and predates RP1. It cannot work on a Pi 5.
- **Whatever apt gives you** may be an ancient packaged version. Don't trust it blindly.

The living project is github.com/WiringPi/WiringPi, maintained since 2024 by GC2, currently v3.x. It supports every Pi board including the Pi 5, driving the RP1 I/O controller directly. The one gap: GCLK (clock-output) functionality is unsupported on Pi 5 because RP1's clock block isn't documented — you won't need it here. There's also a known minor RP1 performance quirk on rapidly alternating read/write operations; irrelevant for these lessons.

The trade you're making versus a raw-kernel approach: wiringPi hands you **one uniform API** across GPIO, PWM, I²C, and SPI. That's less code and less subsystem-juggling — at the cost of leaning on a Pi-specific abstraction layer rather than the kernel's native interfaces. For learning the *electronics* with minimal friction, that's a good trade.

Before you start: full pre-flight setup

Do this once, top to bottom. Skipping a step here is the cause of most "it compiles but nothing happens" sessions.

1 · Confirm your OS and hardware

```
cat /etc/os-release      # Raspberry Pi OS (Debian 12 'bookworm' or newer)
uname -a                 # kernel version
cat /proc/device-tree/model # confirm 'Raspberry Pi 5'
```

2 · Update, then install the toolchain

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y g++ build-essential git i2c-tools cmake
```

3 · Build and install the maintained WiringPi fork

This is the step that replaces your kit's broken bundled libraries. Build from source so you get the current RP1-aware version — not an old packaged one.

```
git clone https://github.com/WiringPi/WiringPi.git
cd WiringPi
./build debian          # builds a .deb for your architecture
mv debian-template/wiringpi-3.*.deb .
sudo apt install ./wiringpi-3.*.deb # installs libwiringPi + the 'gpio' tool
```

Verify the install and confirm it sees a Pi 5:

```
gpio -v      # should report version 3.x
gpio readall # prints the BCM / wPi / physical pin map for your board
```

Pin numbering — decide once, here. `gpio readall` shows three numbering columns: **BCM** (Broadcom GPIO), **wPi** (wiringPi's own scheme), and **Physical** (header position). This document uses **BCM numbering everywhere**, activated by calling `wiringPiSetupGpio()` at the top of every program. That keeps every pin number in these lessons identical to the wiring diagrams and datasheets.

Note this differs from your `blink_led.cpp`, which called plain `wiringPiSetup()` — that selects wiringPi's *own* numbering, where `pin 25` is actually BCM GPIO 26. Mixing the two schemes is the #1 source of "wired correctly, wrong pin toggles" bugs. Pick `wiringPiSetupGpio()` and stay in BCM.

4 · Enable the buses you'll need

```
sudo raspi-config
# Interface Options -> I2C -> Enable   (ADC, expander, LCD, MPU6050, Sense HAT)
# Interface Options -> SPI -> Enable  (only if a lesson calls for SPI)
sudo reboot
```

5 · Grant yourself device access (avoid running everything as root)

```
sudo usermod -aG gpio,i2c,spi,input,video $USER
# log out and back in (or reboot) for group changes to take effect
groups    # verify: gpio i2c spi input video ... appear
```

6 · Sanity-check that the hardware is visible

```
gpio readall      # lines listed, modes shown
i2cdetect -y 1    # after wiring an I2C part, its address appears in the grid
```

7 · Keep the kit's reference material on the Pi

```
git clone --depth 1 \
  https://github.com/Freenove/Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi.git
```

Use its **datasheets and wiring diagrams**. Treat its C code as reference for *what* each circuit does, not *how* to drive it — its examples target old wiringPi/pin numbering. You're re-implementing on the current fork with BCM numbering.

8 · Bench safety (every wiring session)

- Power the Pi off before re-wiring if you're not fully sure of the circuit.
- Every LED gets a current-limiting resistor (220–330Ω). No bare LED across a pin.
- GPIO pins are 3.3V and low-current. Never feed 5V back into a GPIO, and never drive a motor from a Pi pin — use the L293D with external power.
- Double-check GND and 3V3/5V rails before powering on. A reversed rail is how parts die.

9 · The compile command you'll reuse everywhere

```
g++ prog.cpp -o prog -std=c++20 -Wall -Wextra -lwiringPi
# add -lpthread if softPwm/hardware-thread linking complains:
g++ prog.cpp -o prog -std=c++20 -Wall -Wextra -lwiringPi -lpthread
./prog
```

Or with CMake:

```
add_executable(prog prog.cpp)
target_compile_features(prog PRIVATE cxx_std_20)
target_link_libraries(prog PRIVATE wiringPi)
```

The one pattern to internalize: setup once, then drive pins directly

libgpiod made you request and *own* each line (RAII line ownership). wiringPi's model is the opposite and simpler: you call **one setup function once**, and from then on the GPIO pins are

process-wide state you poke directly. There's no per-line ownership object and no automatic release — the pins simply keep whatever mode and value you last set.

```
#include <wiringPi.h>

int main() {
    if (wiringPiSetupGpio() == -1) return 1; // BCM numbering, once

    pinMode(17, OUTPUT);           // configure the pin's mode
    digitalWrite(17, HIGH);        // drive it — no request object needed
    // pin 17 stays HIGH until you change it; nothing auto-releases at scope end
}
```

Ask yourself as you go: *what state is global now? what happens if my program crashes mid-run?* (Answer: the pin holds its last value — there's no automatic cleanup, which is a real difference from libgpod's auto-release.)

Miss RAIL? Layer it on yourself (optional). wiringPi won't give you RAIL, but nothing stops you from adding it where cleanup actually matters — an open file descriptor, a running softPwm channel, an `mmap`. You'll do exactly this for the I²C file descriptor in Lesson 8. That's the honest place for RAIL in a wiringPi program: not around bare pins, but around resources that genuinely need releasing.

Unit I — GPIO, PWM, and I²C

Lesson 1 — Blink an LED

Hardware: 1 LED, 220Ω resistor. GPIO17 → resistor → LED(+), LED(−) → GND. *C++ focus:* the compile/run loop, wiringPi's setup contract, timing.

PREDICT FIRST — What is the minimum that must happen between `HIGH` and `LOW` for you to see a blink? What if you delete both `delay()` calls?

FILL IN THE BLANKS

```

#include <wiringPi.h>
#include <iostream>

constexpr int LED = 17;

int main() {
    if (wiringPiSetupGpio() == -1) {
        std::cerr << "wiringPi init failed\n";
        return 1;
    }
    pinMode(LED, _____);           // which mode?

    for (int i = 0; i < _____; ++i) {
        digitalWrite(LED, _____); // on
        delay(_____);                // milliseconds
        digitalWrite(LED, _____); // off
        delay(_____);
    }
    return 0;
}

```

`delay(ms)` is wiringPi's millisecond sleep; `delayMicroseconds(us)` is the fine-grained one. `std::this_thread::sleep_for(...)` also works if you prefer `<chrono>`, exactly as in your `blink_led.cpp`.

IMPLEMENT / EXTEND — Get a 10-blink loop working. Then, independently, blink **SOS** (… ——— …) using a helper `void flash(int ms)` — no copy-pasting nine blinks. Give the letters distinct gaps so S/O/S are actually distinguishable by eye.

Lesson 2 — Read a button

Hardware: 1 push button. GPIO2 → button → GND, internal pull-up. *C++ focus:* input mode, pull-up bias, edge detection, a tiny state machine.

PREDICT FIRST — With a pull-up enabled, does an *unpressed* button read **HIGH** or **LOW**? What does it read while *pressed*? Predict both before printing.

FILL IN THE BLANKS

```

#include <wiringPi.h>
#include <iostream>

constexpr int BTN = 2;

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    pinMode(BTN, _____);           // which mode?
    pullUpDnControl(BTN, _____);   // which bias? PUD_UP / PUD_DOWN / PUD_OFF

    int prev = _____;             // resting level with a pull-up
    while (true) {
        int now = digitalRead(BTN);
        if (now == _____ && prev == _____) { // detect exactly ONE edge
            std::cout << " _____\n";
        }
        prev = _____;
        delay(5);                       // debounce-ish poll interval
    }
}

```

IMPLEMENT / EXTEND — Print a message only on the **press→release** transition, not while held. Then invert it to fire on **release→press**. Notice how the blanks force you to define exactly one edge.

BIAS GOTCHA — Without `pullUpDnControl(BTN, PUD_UP)` the pin floats and reads garbage. `gpiozero` set this for you invisibly; wiringPi makes you own the decision — just like libgpod did, but through one function call instead of a line-settings builder.

Lesson 3 — Button-controlled LED

Hardware: LED on GPIO17, button on GPIO2. *C++ focus:* independent pins vs. a grouped request; mapping input to output; toggling.

PREDICT FIRST — In libgpod you'd hold both lines in one `request` object. In wiringPi each pin is configured independently with its own `pinMode`. What do you *lose* by not having a single grouped claim, and does it matter for this circuit?

FILL IN THE BLANKS

```

#include <wiringPi.h>

constexpr int BTN = 2;
constexpr int LED = 17;

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    pinMode(BTN, INPUT);
    pullUpDnControl(BTN, PUD_UP);
    pinMode(LED, _____);

    bool led_on = false;
    int prev = HIGH; // resting (pull-up)
    while (true) {
        int now = digitalRead(BTN);
        if (/* press edge */ _____) {
            led_on = _____; // TOGGLE
            digitalWrite(LED, led_on ? ____ : ____);
        }
        prev = now;
        delay(5);
    }
}

```

IMPLEMENT / EXTEND — Make the button toggle the LED on each press (reuse your Lesson 2 edge logic). Then try a "hold = on, release = off" variant and compare which feels cleaner to code.

Lesson 4 — PWM: LED brightness

Hardware: 1 LED on GPIO18. *C++ focus:* duty cycle, and the two kinds of PWM wiringPi gives you.

PREDICT FIRST — With a PWM range of 100, what value gives ~50% brightness? And what's the difference between *software* PWM (any pin, threaded) and *hardware* PWM (dedicated pins, rock-steady)?

FILL IN THE BLANKS — software PWM (works on any pin)

```

#include <wiringPi.h>
#include <softPwm.h>

constexpr int LED = 18;

int main() {
    if (wiringPiSetupGpio() == -1) return 1;

    softPwmCreate(LED, 0, ____);          // (pin, initial, range) -- range 100
    while (true) {
        for (int duty = 0; duty <= 100; ++duty) {    // ramp up
            softPwmWrite(LED, ____);                // set brightness
            delay(____);
        }
        for (int duty = 100; duty >= 0; --duty) {    // ramp down
            softPwmWrite(LED, ____);
            delay(____);
        }
    }
}

```

IMPLEMENT / EXTEND — Ramp the LED smoothly off→full→off. Keep the duty a *variable*, not scattered magic delays. Then rewrite the same ramp using **hardware PWM** and compare the smoothness with your eyes:

```

pinMode(18, PWM_OUTPUT);          // BCM 18 is a hardware-PWM pin
pwmSetMode(PWM_MODE_MS);          // mark-space mode
pwmSetRange(1024);
pwmSetClock(____);                // pick a divisor; compute the resulting frequency
pwmWrite(18, ____);                // 0..1024

```

THE BIG PWM CHANGE — This is where the wiringPi edition diverges most from a raw-libgpiod plan. libgpiod is digital-only, forcing you to bit-bang PWM in a jittery busy loop. wiringPi gives you **two** real options: `softPwm` (any pin, runs each channel in its own thread — smoother than hand-bit-banging, but still software, so some jitter) and **true hardware PWM** via `pwmWrite()` on BCM **12, 13, 18, 19 only**. GPIO18 is one of them. Feel the difference between the two before moving on.

Lesson 5 — RGB LED

Hardware: 1 common-cathode RGB LED, 3 resistors, 3 pins. *C++ focus:* composing three PWM channels; a struct to model color state.

PREDICT FIRST — Magenta = red + blue, no green. Hardware PWM has only **two** channels (BCM 12/18 share channel 0; 13/19 share channel 1). Given that, why must a 3-pin RGB LED use `softPwm` rather than hardware PWM?

FILL IN THE BLANKS


```

#include <wiringPi.h>
#include <softPwm.h>

constexpr int R = 17, G = 27, B = 22;          // adjust to your wiring

struct Color { int r, g, b; };                // each 0..100 duty

void show(const Color& c) {
    softPwmWrite(R, ____);
    softPwmWrite(G, ____);
    softPwmWrite(B, ____);
}

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    softPwmCreate(R, 0, 100);
    softPwmCreate(G, 0, 100);
    softPwmCreate(B, 0, 100);

    Color magenta{____, ____, ____};
    while (true) show(magenta);
}

```

IMPLEMENT / EXTEND — Cycle red→green→blue, then three mixes (magenta, cyan, yellow). One `show(Color)` function — don't inline the three `softPwmWrite` calls everywhere.

THREADED PWM — Each `softPwmCreate` spins up its own thread, so three simultaneous channels behave far better than the flicker you'd get hand-bit-banging three pins in one loop. It's still *software* PWM, though — hardware PWM's two channels simply can't cover three colors.

Lesson 6 — 7-segment display (5161BS)

Hardware: 1× 5161BS 7-seg, resistors, 7–8 pins. *C++ focus:* bit patterns, lookup tables, data→pin mapping.

PREDICT FIRST — Sketch the segment on/off pattern for '3' and '7' by hand. How many pins total do you need for 7 segments?

FILL IN THE BLANKS

```

#include <wiringPi.h>
#include <array>
#include <stdint>

// segment order: a b c d e f g    (1 = lit, for common-cathode)
const std::array<uint8_t, 10> DIGIT = {
    0b_____,    // 0
    0b_____,    // 1
    // ... fill 2..9 yourself
};

const int SEG[7] = {21, 20, 16, 22, 23, 24, 12}; // BCM pins for a..g (adjust)

void write_digit(uint8_t pattern) {
    for (int s = 0; s < 7; ++s) {
        bool on = _____; // extract bit s from pattern
        digitalWrite(SEG[s], on ? HIGH : LOW);
    }
}

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    for (int s = 0; s < 7; ++s) pinMode(SEG[s], OUTPUT);
    // count 0->9 ...
}

```

IMPLEMENT / EXTEND — Count 0→9 with a delay, driven by your lookup table. Defend your choice of `std::array` vs a raw bitmask at review.

POLARITY — Confirm **common-anode vs. common-cathode** — it flips whether a segment lights on `HIGH` or `LOW`. Pinout is in [5161bs.pdf](#).

Lesson 7 — Shift register (74HC595)

Hardware: 1× 74HC595, 8 LEDs, only 3 GPIO pins. *C++ focus:* bit-banging a serial protocol; clocking; latching.

PREDICT FIRST — If you shift out `0b10000000`, does the first or last LED light? Why does bit order depend on your loop direction?

FILL IN THE BLANKS

```

#include <wiringPi.h>
#include <stdint>

constexpr int DATA = 17, CLOCK = 27, LATCH = 22; // SHCP=CLOCK, STCP=LATCH

void shiftOut595(uint8_t value) {
    for (int i = 7; i >= 0; --i) {
        bool bit = _____; // pick bit i of value
        digitalWrite(DATA, bit ? HIGH : LOW);
        digitalWrite(CLOCK, _____); // pulse the clock: rising...
        digitalWrite(CLOCK, _____); // ...then low
    }
    digitalWrite(LATCH, _____); // latch the byte to the outputs
    digitalWrite(LATCH, _____);
}

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    for (int p : {DATA, CLOCK, LATCH}) pinMode(p, OUTPUT);
    // Knight Rider: run one lit LED across all 8 ...
}

```

IMPLEMENT / EXTEND — Prove `shiftOut595` works by running one lit LED across all 8 ("Knight Rider"). Once yours works, compare it to wiringPi's built-in `shiftOut(dataPin, clockPin, order, val)` — you implemented exactly what that library call does.

PURE DIGITAL — No PWM needed; wiringPi toggles these pins perfectly. The '595 latches on clock edges, so timing is forgiving. Truth table in [74HC595.pdf](#).

Lesson 8 — I²C: analog input via ADC (ADS7830 / PCF8591)

Hardware: 1 ADC chip + potentiometer on I²C (SDA/SCL). *C++ focus:* the Pi has no analog input; talking to a chip over a bus; RAI on a file descriptor.

PREDICT FIRST — The Pi 5 can't read analog voltage directly — a hardware fact, not a library limit. Does the ADC hand you back a *voltage* or a *number*?

FILL IN THE BLANKS

```

#include <wiringPiI2C.h>
#include <unistd.h>    // close()
#include <cstdlib>

class I2C {           // thin RAII wrapper over wiringPi's I2C helpers
    int fd;
public:
    explicit I2C(int addr) {
        fd = wiringPiI2CSetup(____);    // opens /dev/i2c-1, selects addr
    }
    ~I2C() { ____ (fd); }                // release the fd (RAII, your job here)

    int readByte()      { return wiringPiI2CRead(fd); }
    int writeByte(uint8_t v) { return wiringPiI2CWrite(fd, v); }
    int readReg8(uint8_t reg) { return wiringPiI2CReadReg8(fd, reg); }
};

int main() {
    I2C adc(0x__);                // address from i2cdetect -y 1
    while (true) {
        adc.writeByte(/* channel command byte */ ____);    // ADS7830-style
        int value = adc.readByte();    // 0..255
        // print value ...
    }
}

```

IMPLEMENT / EXTEND — Print the 0–255 value as you turn the pot. Keep the chip wrapped in this class — **you reuse it in Lessons 9, 10, and 14**. A clean class here saves you four times over.

ONE LIBRARY, STILL THE I²C SUBSYSTEM — `wiringPiI2C*` wraps the same `/dev/i2c-1` kernel device libgpiod-based plans reached for directly — so now one library (`wiringPi`) covers GPIO, PWM, *and* I²C. The RAII wrapper is still worth writing: `wiringPiI2CSetup` hands you a raw fd, and you own closing it. Find the address first with `i2cdetect -y 1`; the ADS7830 and PCF8591 use different command bytes — see their datasheets.

Lesson 9 — I²C I/O expander (PCF8574) + LCD1602

Hardware: PCF8574 backpack driving a 16×2 LCD. *C++ focus:* reusing your I²C class; device init sequences and timing.

PREDICT FIRST — The LCD needs a specific init sequence and a 4-bit nibble protocol. What goes wrong if you skip one of the datasheet delays?

FILL IN THE BLANKS

```
// Reuse the I2C class from Lesson 8.
void lcd_write4(I2C& lcd, uint8_t nibble, bool is_data); // you write this

void lcd_send(I2C& lcd, uint8_t value, bool is_data) {
    uint8_t hi = value & 0xF0;
    uint8_t lo = _____; // low nibble, shifted up
    lcd_write4(lcd, hi, is_data);
    lcd_write4(lcd, lo, is_data);
}

void lcd_init(I2C& lcd) {
    // follow the datasheet timing EXACTLY -- use delay()/delayMicroseconds():
    _____
    _____
}
```

IMPLEMENT / EXTEND — Display "Hello". This is your first real build-on-previous-work lesson — a clean Lesson 8 class makes it far easier. (wiringPi also ships an `lcd.h` helper with `lcdInit()`; do it by hand first, then peek.)

TIMING OVER RAW GPIO — Sequencing and delays matter more than pin toggling here. See [PCF8574_datasheet.pdf](#) and [LCD1602.pdf](#); don't rush the init.

Lesson 10 — MPU6050 accelerometer + gyro

Hardware: MPU6050 6-axis IMU over I²C. *C++ focus:* multi-byte registers; signed 16-bit assembly; two's complement.

PREDICT FIRST — Each axis is two bytes you must combine into a *signed* 16-bit value. What breaks if you forget it's signed?

FILL IN THE BLANKS

```
// Reuse the I2C class from Lesson 8. MPU6050 default address 0x68.
#include <stdint>

int16_t axis(uint8_t hi, uint8_t lo) {
    return (int16_t)( ( _____ << 8) | _____ ); // assemble, keep the sign
}

int main() {
    I2C imu(0x68);
    imu.writeReg8(0x6B, 0x00); // wake the device (PWR_MGMT_1)

    while (true) {
        uint8_t xh = imu.readReg8(0x3B), xl = imu.readReg8(0x3C);
        int16_t ax = axis(_____, _____);
        double g = ax / _____; // scale for +/-2g range (datasheet)
        // print all three axes ...
    }
}
```

(Add a `writeReg8` method to your I2C class — `wiringPiI2CWriteReg8(fd, reg, val)` — the mirror of `readReg8`.)

IMPLEMENT / EXTEND — Print g-values for all three axes as you tilt the board. Reuse your I²C class a third time; the register map is your spec.

BIT ASSEMBLY, NOT GPIO — The C++ meat here is `(hi << 8) | lo` plus two's complement, not pin wrangling. Understand every operator. See [MPU6050_Register_Map_and_Descriptions.pdf](#).

Lesson 11 — Motor + driver (L293D)

Hardware: L293D H-bridge, DC motor, external power. *C++ focus:* direction logic, PWM speed, electrical safety.

PREDICT FIRST — Predict the two IN-pin combinations for forward, reverse, and brake. What does `IN1 == IN2` do, and what's the one electrical rule you must never break?

FILL IN THE BLANKS

```
#include <wiringPi.h>
#include <softPwm.h>

constexpr int IN1 = 17, IN2 = 27, EN = 18; // EN on a PWM-capable pin

void drive(bool forward, int duty) { // duty 0..100
    digitalWrite(IN1, forward ? HIGH : LOW);
    digitalWrite(IN2, forward ? LOW : _____);
    softPwmWrite(EN, _____); // speed via PWM on the enable pin
}

int main() {
    if (wiringPiSetupGpio() == -1) return 1;
    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    softPwmCreate(EN, 0, 100);
    // forward, then reverse, ramping speed ...
    // document in a comment: IN1=____, IN2=____ means ____ (and why external power is
    mandatory)
}
```

IMPLEMENT / EXTEND — Drive forward and reverse, then add PWM speed on `EN`. For steadier speed, swap `softPwm` on `EN` for hardware PWM (put `EN` on BCM 12/13/18/19). Combine direction logic with your Lesson 4 PWM understanding.

POWER SAFETY — **NEVER** power the motor from the Pi's 3.3V/5V pins; use an external supply through the L293D, sharing a common ground. PWM speed on `EN` will make the motor whine at low duty — that's normal. See [L293D.pdf](#).

Unit II — Sense HAT

The Sense HAT is a separate add-on board (8×8 RGB LED matrix, environmental sensors, IMU, 5-way joystick) that stacks on the 40-pin header.

wiringPi DOESN'T REACH MOST OF THIS — Important scope note: the Sense HAT bypasses wiringPi almost entirely. The **LED matrix is a Linux framebuffer**, the **joystick is an evdev input device**, and the **sensors are on I²C**. So of the four lessons below, only Lesson 14 (sensors) can reuse your `wiringPiI2C` class — the matrix and joystick drop straight to kernel interfaces. This is the honest edge of the "one uniform API" trade: wiringPi covers GPIO/PWM/I²C/SPI, but not framebuffers or input events.

Sense HAT setup (once):

```
sudo apt install -y sense-hat librtimulib-dev rtimulib-utils
# ensure the LED driver loads -- add to /boot/firmware/config.txt if needed:
# dtoverlay=rpi-sense
sudo reboot
# find the LED matrix framebuffer (its name is 'RPi-Sense FB'):
for fb in /dev/fb*; do echo -n "$fb: "; cat /sys/class/graphics/${fb##*/}/name; done
```

Lesson 12 — LED matrix: light one pixel (framebuffer + RGB565)

Hardware: Sense HAT 8×8 RGB matrix. *C++ focus:* `mmap`, the framebuffer device, RGB565 bit-packing.

PREDICT FIRST — 64 pixels × 2 bytes = 128 bytes. In RGB565, red uses 5 bits, green 6, blue 5. How do you pack `(r,g,b)` into one `uint16_t`? Work out the shifts before coding.

FILL IN THE BLANKS

```
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <cstring>
#include <stdint>

uint16_t rgb565(uint8_t r, uint8_t g, uint8_t b) {
    return ((r >> 3) << ____ ) | ((g >> 2) << ____ ) | (b >> 3);
}

int main() {
    int fd = open("/dev/fb_", O_RDWR); // the RPi-Sense FB node you detected
    auto* px = (uint16_t*) mmap(nullptr, 128,
                                PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    std::memset(px, 0, 128); // clear
    px[____] = rgb565(255, 0, 0); // light pixel (x=0, y=0) red
    // leave it lit briefly, then clear, munmap, close
}
```

IMPLEMENT / EXTEND — Light one red pixel, then fill the whole matrix one color, then draw a diagonal. Index math: pixel (x,y) lives at $y*8 + x$.

DETECT, DON'T HARD-CODE — The matrix is `/dev/fb0` on some setups and `/dev/fb1` on others (depends on whether a display is attached). Detect the node whose name is `RPi-Sense FB`. If colors look wrong, revisit your 5/6/5 shifts before suspecting endianness.

Lesson 13 — Joystick: read input via evdev

Hardware: Sense HAT 5-way mini joystick. *C++ focus:* the evdev input interface; event structs; blocking reads.

PREDICT FIRST — The joystick reports as a keyboard (arrow keys + Enter) on `/dev/input/event*`. Which fields of `struct input_event` tell you the direction and whether it's a press vs. release?

FILL IN THE BLANKS

```
#include <linux/input.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("/dev/input/event_", O_RDONLY); // the Sense HAT joystick node
    struct input_event ev;
    while (read(fd, &ev, sizeof(ev)) == sizeof(ev)) {
        if (ev.type == EV_KEY && ev.value == ____ ) { // 1=press, 2=repeat, 0=release
            switch (ev.code) {
                case KEY_UP: /* ____ */ break;
                case KEY_DOWN: /* ____ */ break;
                case KEY_LEFT: case KEY_RIGHT: case KEY_ENTER: /* ... */ break;
            }
        }
    }
}
```

IMPLEMENT / EXTEND — Print the direction on each press. Then combine with Lesson 12: move a lit pixel around the matrix with the joystick (clamp to 0-7).

FIND THE RIGHT NODE — `cat /proc/bus/input/devices` and look for "Raspberry Pi Sense HAT Joystick". Its `event*` number can change across reboots — detect by name for robustness.

Lesson 14 — Environmental sensors: temperature, pressure, humidity

Hardware: Sense HAT pressure + humidity/temperature sensors (I²C). *C++ focus:* reusing your I²C class on new devices; datasheet register reads.

PREDICT FIRST — These sensors sit on the same I²C bus you already used. Why can you reuse most of your Lesson 8 class unchanged? What's the only part that changes per sensor?

FILL IN THE BLANKS

```
// Reuse the I2C RAII class from Lesson 8 -- new address + registers.
// Read raw temperature, then apply the datasheet's conversion.
I2C sensor(0x__); // confirm with i2cdetect -y 1
int raw = /* read the sensor's data registers */ ____;
double temp_c = ____; // per datasheet calibration
// print temp_c ...
```

IMPLEMENT / EXTEND — Print temperature once, then loop once/second. Then show it on the LED matrix (Lesson 12) as a color shifting blue→red with temperature — your first sensor→display pipeline.

KNOW YOUR BOARD — Confirm addresses with `i2cdetect -y 1`. The exact sensor chips differ between Sense HAT v1 and v2, so read the datasheet for *your* board rather than assuming registers.

Lesson 15 — IMU orientation with RTIMULib (capstone)

Hardware: Sense HAT 9-DoF IMU (accel + gyro + magnetometer). *C++ focus:* linking a real C++ library; sensor fusion you don't hand-roll.

PREDICT FIRST — Raw accel/gyro/mag needs *fusion* to give stable orientation. Why is hand-rolling that (vs. Lesson 10's raw reads) a bad use of your time here? What does RTIMULib do that you shouldn't reinvent?

FILL IN THE BLANKS

```

#include "RTIMULib.h"
#include <thread>
#include <chrono>
#include <numbers>

int main() {
    RTIMUSettings* settings = new RTIMUSettings("RTIMULib");
    RTIMU* imu = RTIMU::createIMU(settings);
    imu->IMUInit();

    while (true) {
        if (imu->IMURead()) {
            RTIMU_DATA d = imu->getIMUData();
            double roll = d.fusionPose.x() * 180.0 / std::numbers::pi;
            double pitch = _____;
            double yaw = _____;
            // print roll/pitch/yaw ...
        }
        std::this_thread::sleep_for(
            std::chrono::milliseconds(imu->IMUGetPollInterval()));
    }
}

```

IMPLEMENT / EXTEND — Print roll/pitch/yaw as you tilt the board. **Capstone:** combine everything — use the IMU to roll a lit pixel around the matrix (Lesson 12) like a ball in a tray, reset with the joystick (Lesson 13).

LINKING — `g++ imu.cpp -o imu -std=c++20 -lRTIMULib`. Copy `/etc/RTIMULib.ini` into your working directory for calibrated data — the examples look for it in the current folder. Note `std::numbers::pi` (C++20) replaces the old `M_PI`.

Where this leaves you

By Lesson 15 you'll have written, from scratch: a reusable I²C class with RAIL on a file descriptor, a bit-banged serial protocol, signed-integer assembly from raw registers, direct framebuffer graphics with hand-packed RGB565, and evdev input handling — all driven through wiringPi where it reaches, and through raw kernel interfaces where it doesn't.

The recurring lessons worth naming

- **One library, one API.** wiringPi unifies GPIO, PWM, I²C, and SPI — simpler than juggling separate kernel subsystems, at the cost of a Pi-specific abstraction. The maintained fork is what makes that possible on the Pi 5.
- **Two flavors of PWM.** `softPwm` works on any pin and is threaded but still software (some jitter). Hardware PWM (`pwmWrite`) is steady but limited to BCM 12/13/18/19 — two channels. Choose per lesson.
- **The Pi has no analog input.** Analog sensors need an ADC over I²C (Lesson 8+).

- **wiringPi has edges.** The Sense HAT's matrix (framebuffer) and joystick (evdev) live outside wiringPi entirely — you drop to kernel interfaces there. Knowing where the abstraction stops is part of the skill.
- **Datasheets are the real spec.** From Lesson 6 on, the kit's/board's PDFs beat any tutorial.
- **Reuse compounds.** One clean `wiringPiI2C` wrapper serves Lessons 8, 9, 10, and 14. Sloppy code taxes you four times.
- **Use the right wiringPi.** Never fall back to `apt`'s ancient package or Gordon's deprecated 2.5 — build the GC2 fork from source.

Your rule, restated: the blanks exist so you write the parts that teach. If you catch yourself wanting the filled-in version instead of wanting to *understand* it, stop and ask what that shortcut buys you. Usually nothing. Bring the code you wrote — broken or working — and that's where the real review happens.